

Dynamic AI-Driven UI Generation System

Integrated Product Requirements & Implementation Plan

Project: Consumer IQ - GTM Launch
Project Key: CGL
Version: 2.0
Date: October 9, 2025
Status: Ready for Development

Executive Summary

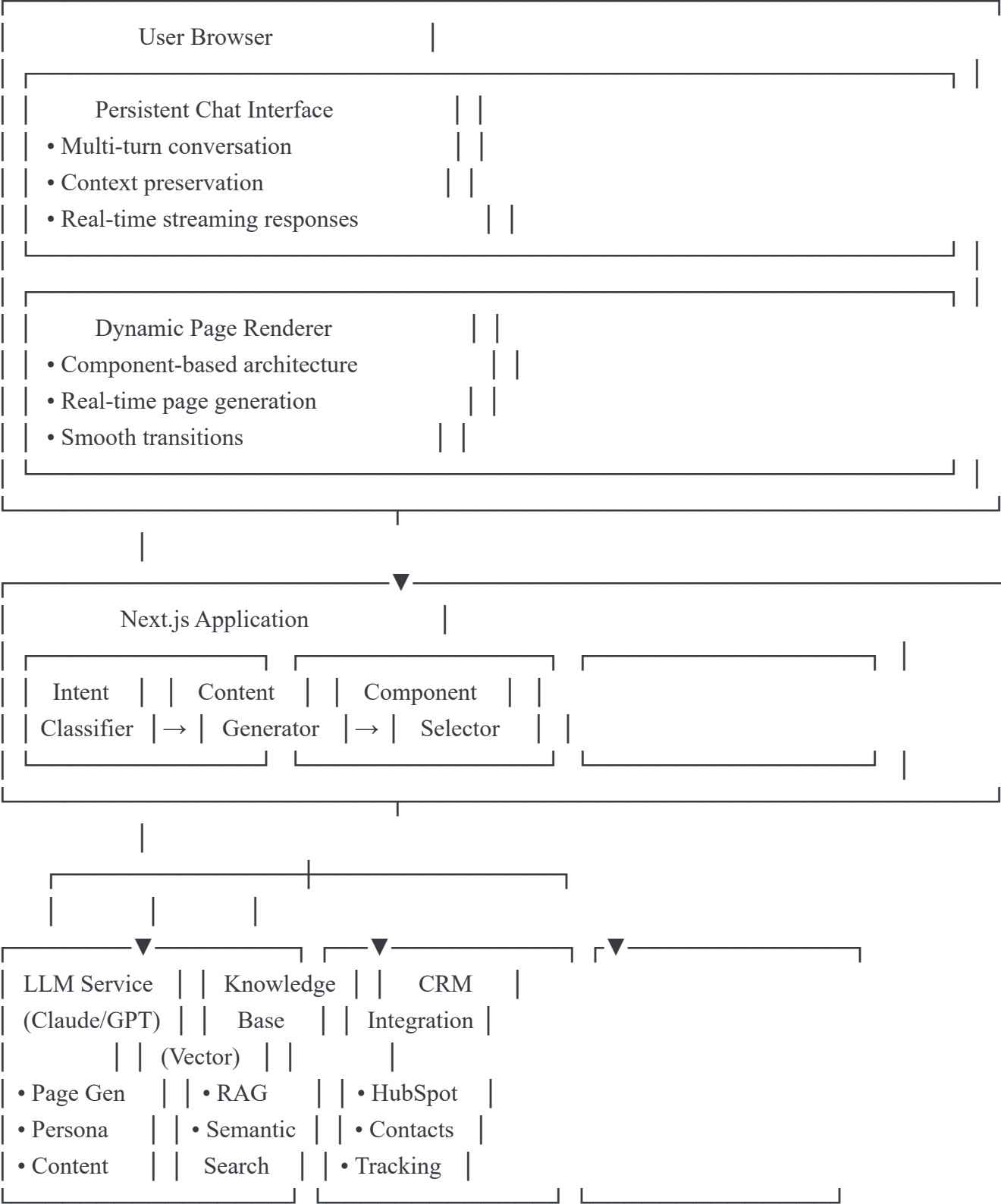
This document outlines the complete implementation plan for Consumer IQ's Dynamic AI-Driven UI Generation System - a conversation-first web application where an AI agent dynamically generates both content and user interfaces in response to natural language queries. The system eliminates traditional static pages, enabling users to explore information through natural language while maintaining enterprise-grade security, performance, and scalability.

Table of Contents

- [1. System Architecture Overview](#)
- [2. Epic 1: Core Chat & Page Generation Engine](#)
- [3. Epic 2: RAG Knowledge Base & Content Management](#)
- [4. Epic 3: Dynamic Component Library](#)
- [5. Epic 4: Persona Detection & Personalization](#)
- [6. Epic 5: Lead Capture & CRM Integration](#)
- [7. Epic 6: Security & Compliance](#)
- [8. Epic 7: Observability & Analytics](#)
- [9. Epic 8: Performance & Optimization](#)
- [10. Implementation Phases](#)

System Architecture Overview





Epic 1: Core Chat & Page Generation Engine

Story 1.1: Build Persistent Chat Interface

Priority: P0 (Critical)
Story Points: 13
Labels: frontend, chat, core

Description:

As a user, I want to interact with an AI agent through a persistent chat interface that remains accessible while navigating dynamically generated pages.

Acceptance Criteria:

- Chat interface visible on all pages (floating widget or side panel)
- Support text input with multi-line capability
- Display conversation history with scroll
- Maintain context across page generations
- Support "Clear conversation" action
- Responsive design (mobile, tablet, desktop)
- Smooth animations and transitions
- Loading states during AI processing
- Error states with retry capability

Technical Requirements:

- React component with state management (Zustand/Context)
 - WebSocket or SSE for streaming responses
 - Local storage for session persistence
 - Accessibility: ARIA labels, keyboard navigation
 - Performance: Virtualized scrolling for long conversations
-

Story 1.2: Implement Intent Classification System

Priority: P0 (Critical)

Story Points: 8

Labels: backend, ai, core

Description:

As the system, I need to classify user intent from natural language queries to determine appropriate content and UI generation strategy.

Acceptance Criteria:

- Classify intents into categories:
 - Informational (learn about features)
 - Comparative (compare options/plans)
 - Navigational (find specific content)
 - Transactional (take action/get started)
 - Analytical (view data/insights)
- 90% accuracy on test dataset
- Handle ambiguous queries gracefully
- Support 20+ distinct intent subtypes
- Return confidence scores
- Sub-200ms classification time

Technical Requirements:

- LLM-based classification with structured output
- Fallback intent classification rules
- Intent taxonomy configuration file
- Unit tests with example queries
- A/B testing framework for improvements

Implementation Tasks:

1. Define intent taxonomy and examples

2. Create LLM prompt for classification
 3. Build intent classifier service
 4. Add confidence scoring logic
 5. Implement fallback rules
 6. Write comprehensive tests
 7. Document intent types and examples
-

Story 1.3: Build Dynamic Page Generator

Priority: P0 (Critical)

Story Points: 21

Labels: backend, ai, core

Description:

As the system, I need to generate complete page specifications (content + components + layout) based on user queries and intent.

Acceptance Criteria:

- Generate page spec from intent and context
- Select appropriate components for content type
- Create cohesive layouts (3-7 components per page)
- Include all necessary content (headings, body, CTAs)
- Support page types: landing, feature, comparison, dashboard
- Generate pages in <2 seconds (90th percentile)
- Handle errors gracefully with fallbacks
- Maintain consistency across related pages

Technical Requirements:

- Page specification schema (TypeScript interfaces)
- Component registry with metadata
- Layout engine for component arrangement
- Content generation pipeline
- Caching for common patterns
- Streaming generation for responsiveness

Implementation Tasks:

1. Define page specification schema
 2. Create component registry and selector
 3. Build layout generation algorithm
 4. Implement content population logic
 5. Add caching layer for performance
 6. Write integration tests
 7. Optimize for latency and quality
-

Story 1.4: Implement Page Renderer

Priority: P0 (Critical)

Story Points: 13

Labels: frontend, core

Description:

As a user, I want to see dynamically generated pages render smoothly with all components functioning correctly.

Acceptance Criteria:

- Render pages from specifications
- Support all component types (20+ components)
- Responsive layouts (mobile, tablet, desktop)
- Smooth transitions between pages
- Loading states during generation
- Error states with recovery options
- Support for interactive components
- Maintain scroll position when appropriate

Technical Requirements:

- Dynamic component loading
- Layout system (Grid, Flexbox)
- Animation library (Framer Motion)
- Error boundaries for component failures
- Performance monitoring (< 1s render time)

Implementation Tasks:

1. Create dynamic component loader
 2. Build layout rendering system
 3. Add transition animations
 4. Implement loading/error states
 5. Add responsive breakpoints
 6. Write component tests
 7. Performance optimization
 8. Accessibility testing
-

Story 1.5: Context Management System

Priority: P0 (Critical)

Story Points: 8

Labels: backend, core

Description:

As the system, I need to maintain conversation context across multiple turns to provide coherent, contextual responses.

Acceptance Criteria:

- Track conversation history (messages, pages)
- Reference previous questions and answers
- Support context-aware follow-ups ("Show me more details")
- Handle context reset ("Start over")
- Store user preferences and selections
- Maintain context for up to 10 turns minimum
- Context persists across session (optional)

Technical Requirements:

- Session storage mechanism (Redis/in-memory)
- Context compression for long conversations
- Context relevance scoring
- Context serialization/deserialization
- TTL management for sessions

Implementation Tasks:

1. Design context schema
2. Implement session storage

- 3. Build context retrieval logic
- 4. Add context compression
- 5. Implement context reset
- 6. Write tests for context flows
- 7. Add monitoring and debugging

Epic 2: RAG Knowledge Base & Content Management

Story 2.1: Vector Database Setup

Priority: P0 (Critical)
Story Points: 8
Labels: backend, infrastructure, rag

Description:
As the system, I need a vector database to store and retrieve product knowledge using semantic search.

Acceptance Criteria:

- Deploy vector database (Supabase pgvector/Pinecone)
- Support embedding storage and retrieval
- Implement k-NN semantic search
- Handle 10,000+ document chunks
- Sub-500ms query latency
- Support metadata filtering
- Backup and recovery procedures

Technical Requirements:

- Vector database deployment
- Embedding dimension: 1536 (OpenAI) or 768 (other)
- Index optimization for performance
- Connection pooling
- Error handling and retries

Implementation Tasks:

- 1. Choose vector database solution
- 2. Set up database and schema
- 3. Configure indexing strategy
- 4. Implement connection client
- 5. Add query optimization
- 6. Test with real data volume
- 7. Document setup and operations

Story 2.2: Content Embedding Pipeline

Priority: P0 (Critical)
Story Points: 13
Labels: backend, rag, content

Description:
As a content manager, I need product content automatically embedded and indexed so the AI can retrieve relevant information.

Acceptance Criteria:

- Chunk documents into 200-500 token pieces
- Preserve context in chunks (include document metadata)
- Generate embeddings for all chunks
- Handle various content types (pages, FAQs, docs)
- Support batch processing (100+ docs)
- Track embedding costs
- Handle embedding API rate limits

Technical Requirements:

- Document chunking strategy
- Embedding API integration (OpenAI/Anthropic)
- Batch processing for efficiency
- Error handling and retries
- Cost tracking and monitoring

Implementation Tasks:

1. Design chunking strategy
 2. Implement document parser
 3. Build embedding generation
 4. Add batch processing
 5. Implement error handling
 6. Track costs and performance
 7. Write tests with various content types
-

Story 2.3: RAG Retrieval System

Priority: P0 (Critical)
Story Points: 13
Labels: backend, ai, rag

Description:
As the system, I need to retrieve relevant context from the knowledge base to augment AI responses with accurate product information.

Acceptance Criteria:

- Semantic search based on user queries
- Return top-k relevant chunks (k=3-5)
- Include relevance scores
- Filter by metadata (content type, persona)
- Re-rank results for quality
- Sub-500ms retrieval time
- Fallback for no results found
- Log retrieval metrics

Technical Requirements:

- Query embedding generation
- Vector similarity search
- Result re-ranking algorithm
- Metadata filtering
- Response formatting
- Cache for common queries

Implementation Tasks:

1. Implement query embedding
 2. Build vector search function
 3. Add metadata filtering
 4. Implement re-ranking
 5. Add caching layer
 6. Optimize for latency
 7. Write comprehensive tests
 8. Monitor retrieval quality
-

Story 2.4: CMS to Vector DB Reindex Pipeline

Priority: P1 (High)

Story Points: 21

Labels: backend, content, infrastructure

Description:

As a content manager, I want CMS updates to automatically refresh the vector store so the Agent always has the latest information.

Acceptance Criteria:

- CMS publish triggers reindex webhook
- Fetch updated content from CMS
- Chunk and embed content
- Upsert to vector database
- Handle content deletions
- Support incremental updates (< 30 seconds)
- Support full reindex (entire knowledge base)
- Error notifications for failures
- Logging and status tracking

Technical Requirements:

- Webhook endpoint (Next.js API route)
- CMS API integration
- Queue system for processing
- Idempotent operations
- Webhook signature validation
- Alerting system

Implementation Tasks:

1. Design webhook architecture
 2. Implement webhook listener
 3. Build CMS content fetcher
 4. Create chunking pipeline
 5. Implement embedding generation
 6. Build vector DB upsert logic
 7. Add error handling and retries
 8. Implement logging and monitoring
 9. Test incremental vs full reindex
 10. Write integration tests
-

Story 2.5: Initial Knowledge Base Population

Priority: P0 (Critical)

Story Points: 8

Labels: backend, content, data

Description:

As a system administrator, I need to populate the initial knowledge base with product content to enable AI responses.

Acceptance Criteria:

- Import product documentation
- Import feature descriptions
- Import use cases and examples
- Import pricing information
- Import FAQ content
- Verify all content indexed correctly
- Test retrieval accuracy
- Document content structure

Technical Requirements:

- Content import scripts
- Data validation
- Bulk upload capabilities
- Progress tracking
- Rollback mechanism

Implementation Tasks:

1. Collect all content sources
2. Standardize content format
3. Create import scripts
4. Validate content structure
5. Execute bulk import
6. Verify indexing
7. Test retrieval quality
8. Document content taxonomy

Epic 3: Dynamic Component Library

Story 3.1: Core Content Components

Priority: P0 (Critical)

Story Points: 21

Labels: frontend, components, core

Description:

As a developer, I need a library of content components that can be dynamically rendered based on page specifications.

Acceptance Criteria:

- Hero section (3 variants)
- Feature grid (2-4 columns)
- Benefit cards (grid/carousel/stack)
- Use case displays
- Text blocks with formatting
- Quote/testimonial cards
- FAQ accordion
- Step-by-step guide

- All components responsive
- Consistent styling (Tailwind)

Technical Requirements:

- React functional components
- TypeScript interfaces for props
- shadcn/ui base components
- Storybook for documentation
- Unit tests for each component

Implementation Tasks:

1. Define component specifications
 2. Create TypeScript interfaces
 3. Build Hero component variants
 4. Build Feature Grid component
 5. Build Benefit Cards component
 6. Build Use Case component
 7. Build Testimonial component
 8. Build FAQ Accordion
 9. Build Step Guide component
 10. Add responsive styling
 11. Write Storybook stories
 12. Write component tests
-

Story 3.2: Data Visualization Components

Priority: P1 (High)

Story Points: 13

Labels: frontend, components, data-viz

Description:

As a user, I want to see data visualizations for analytics and metrics when I ask data-related questions.

Acceptance Criteria:

- Chart component (line, bar, pie, area)
- Data table with sorting/filtering
- Stat cards with trends
- Dashboard layout component
- Configurable colors and styling
- Responsive design
- Loading states
- Error states

Technical Requirements:

- Recharts or Chart.js integration
- TypeScript types for data
- Accessibility for charts
- Performance for large datasets

Implementation Tasks:

1. Choose charting library
2. Build Chart component wrapper
3. Build Data Table component
4. Build Stat Cards component

5. Build Dashboard layout
 6. Add responsive breakpoints
 7. Implement loading/error states
 8. Write component tests
 9. Document data formats
-

Story 3.3: Interactive Components

Priority: P1 (High)
Story Points: 13
Labels: frontend, components, interactive

Description:
As a user, I want interactive components for forms, calculators, and comparisons to engage with the content.

- Acceptance Criteria:**
- Form builder with validation
 - Calculator component
 - Comparison table
 - Tabs component
 - Interactive filter component
 - All components functional
 - Form submission handling
 - Validation with error messages

- Technical Requirements:**
- React Hook Form integration
 - Zod validation schemas
 - Server Actions for submission
 - Loading states for async operations

- Implementation Tasks:**
1. Build Form Builder component
 2. Implement form validation
 3. Build Calculator component
 4. Build Comparison Table
 5. Build Tabs component
 6. Build Filter component
 7. Add Server Actions
 8. Write validation tests
 9. Document component APIs
-

Story 3.4: Component Registry & Selector

Priority: P0 (Critical)
Story Points: 8
Labels: backend, components, core

Description:
As the system, I need a component registry and selection algorithm to choose appropriate components based on content and intent.

- Acceptance Criteria:**
- Register all components with metadata

- Selection algorithm based on:
 - Content type
 - User intent
 - Amount of information
 - Page context
- Return 90% appropriate components
- Support component combinations
- Handle missing components gracefully

Technical Requirements:

- Component metadata schema
- Selection algorithm
- Scoring system for components
- Configuration file for rules

Implementation Tasks:

1. Define component metadata schema
2. Create component registry
3. Build selection algorithm
4. Add scoring logic
5. Implement fallback logic
6. Write selection tests
7. Document selection rules

Epic 4: Persona Detection & Personalization

Story 4.1: Persona Classification System

Priority: P1 (High)

Story Points: 13

Labels: backend, ai, personalization

Description:

As the system, I need to detect user personas (supplier, distributor, competitor, customer) to personalize content and experiences.

Acceptance Criteria:

- Classify persona from conversation patterns
- Support personas: supplier, distributor, competitor, end customer
- Return confidence scores (0-1)
- Update classification as conversation progresses
- Use classification for content selection
- Log persona detection for analysis

Technical Requirements:

- LLM-based classification
- Pattern matching rules
- Confidence scoring
- Incremental updates

Implementation Tasks:

1. Define persona taxonomy
 2. Create classification prompt
 3. Build persona classifier
 4. Add confidence scoring
 5. Implement incremental updates
 6. Test with conversation examples
 7. Document persona definitions
-

Story 4.2: Content Personalization Engine

Priority: P1 (High)

Story Points: 13

Labels: backend, ai, personalization

Description:

As a user, I want to see content personalized to my role and interests based on detected persona and conversation history.

Acceptance Criteria:

- Adjust content depth by persona
- Highlight relevant features by persona
- Customize CTAs by persona
- Adjust technical language level
- Maintain consistency within session
- Override with explicit user preferences

Technical Requirements:

- Persona-based content templates
- Content filtering rules
- Dynamic CTA generation
- Preference storage

Implementation Tasks:

1. Define personalization rules
 2. Create persona content mappings
 3. Build content filter
 4. Implement dynamic CTAs
 5. Add preference overrides
 6. Test personalization accuracy
 7. Document personalization logic
-

Story 4.3: Persona Analytics Dashboard

Priority: P2 (Medium)

Story Points: 8

Labels: frontend, analytics, personalization

Description:

As a marketing stakeholder, I want to see persona distribution and engagement metrics to understand our audience.

Acceptance Criteria:

- Show persona distribution (pie chart)
- Track persona conversion rates
- Display common questions by persona
- Show engagement metrics by persona

- Filter by date range
- Export data

Technical Requirements:

- Analytics data aggregation
- Visualization components
- Real-time or near-real-time updates

Implementation Tasks:

1. Design dashboard layout
2. Build data aggregation queries
3. Create visualization components
4. Implement filtering
5. Add export functionality
6. Test with real data
7. Document metrics

Epic 5: Lead Capture & CRM Integration

Story 5.1: Email Capture Flow

Priority: P1 (High)
Story Points: 8
Labels: frontend, backend, crm

Description:
As a sales stakeholder, I want to capture visitor emails during high-engagement moments to build our lead database.

Acceptance Criteria:

- Trigger email capture after engagement indicators
- Present friendly email form
- Validate email format
- Show success confirmation
- Don't repeatedly ask same user
- Support "No thanks" option
- Log capture attempts

Technical Requirements:

- Email validation (Zod schema)
- Server Action for submission
- Cookie/storage for tracking
- Integration with CRM upsert

Implementation Tasks:

1. Design email capture UI
2. Implement trigger logic
3. Build email form component
4. Add validation
5. Create Server Action
6. Implement tracking cookie
7. Test capture flow

Story 5.2: CRM Integration (HubSpot/Salesforce)

Priority: P1 (High)

Story Points: 13

Labels: backend, crm, integration

Description:

As a sales stakeholder, I want captured emails automatically added to our CRM with rich context so we can follow up effectively.

Acceptance Criteria:

- Choose CRM platform (HubSpot, Salesforce, Pipedrive)
- Define contact schema:
 - Email (required)
 - Persona type and confidence
 - Session ID
 - Pages visited
 - Questions asked
 - Timestamps
- Upsert contacts (create or update)
- Handle API errors with retries
- Log CRM operations
- Send confirmation email to visitor

Technical Requirements:

- CRM API client wrapper
- Idempotent upsert logic
- Error handling and retries
- Email service integration (SendGrid/Postmark)

Implementation Tasks:

1. Choose CRM platform
 2. Design contact schema
 3. Build CRM API client
 4. Implement upsert logic
 5. Add metadata mapping
 6. Build email service integration
 7. Create email template
 8. Implement error handling
 9. Add logging
 10. Write integration tests
-

Story 5.3: Conversation Summary Generation

Priority: P2 (Medium)

Story Points: 8

Labels: backend, ai, crm

Description:

As a visitor, I want to receive an email summary of my conversation so I have a reference of what I learned.

Acceptance Criteria:

- Generate conversation summary with AI
- Include key points discussed
- List relevant resources/pages
- Add appropriate next steps
- Format as professional email
- Send after email capture
- Support unsubscribe

Technical Requirements:

- LLM-based summarization
- HTML email template
- Email delivery tracking

Implementation Tasks:

1. Create summary generation prompt
2. Build summary generator
3. Design email template
4. Implement email sending
5. Add delivery tracking
6. Test with various conversations
7. Ensure GDPR compliance

Epic 6: Security & Compliance

Story 6.1: Input Validation & Sanitization

Priority: P0 (Critical)
Story Points: 8
Labels: backend, security

Description:
As a security stakeholder, I need all user inputs validated and sanitized to prevent injection attacks and ensure data integrity.

Acceptance Criteria:

- Implement Zod schemas for all Server Actions
- Validate email format, message length, field types
- Sanitize text inputs (strip HTML tags)
- Escape special characters
- Return field-level error messages
- Test with malicious inputs
- Document validation rules

Technical Requirements:

- Zod validation library
- Input sanitization utility
- XSS prevention
- SQL injection prevention

Implementation Tasks:

1. Create validation schemas

2. Implement sanitization utility
 3. Add to all Server Actions
 4. Write validation tests
 5. Test with attack vectors
 6. Document validation rules
-

Story 6.2: Rate Limiting Implementation

Priority: P0 (Critical)

Story Points: 13

Labels: backend, security, infrastructure

Description:

As a security stakeholder, I need rate limiting to prevent abuse and ensure service availability for legitimate users.

Acceptance Criteria:

- Per-IP rate limit: 1000 requests/hour
- Per-session rate limit: 100 requests/hour
- Chat rate limit: 10 messages/minute
- Return 429 status when exceeded
- Include Retry-After header
- Log rate limit violations
- Whitelist internal IPs

Technical Requirements:

- Rate limiting solution (Upstash/Vercel/Redis)
- Middleware implementation
- Distributed rate limiting (for scale)

Implementation Tasks:

1. Choose rate limiting solution
 2. Implement rate limit middleware
 3. Configure limits by endpoint
 4. Add violation logging
 5. Implement whitelist
 6. Test with simulated traffic
 7. Monitor in production
-

Story 6.3: Prompt Injection Protection

Priority: P0 (Critical)

Story Points: 8

Labels: backend, security, ai

Description:

As a security stakeholder, I need protection against prompt injection attacks that attempt to manipulate the AI system.

Acceptance Criteria:

- Detect injection patterns:
 - "Ignore previous instructions"
 - "Disregard rules"
 - System prompt extraction attempts
- Block requests with high injection score
- Log injection attempts

- Return generic error without revealing detection
- Balance false positives vs false negatives

Technical Requirements:

- Pattern matching algorithm
- Injection scoring system
- Logging and alerting

Implementation Tasks:

1. Research injection techniques
 2. Create detection patterns
 3. Build injection detector
 4. Implement blocking logic
 5. Add logging
 6. Test with known attacks
 7. Monitor and update patterns
-

Story 6.4: Security Headers & CORS

Priority: P1 (High)

Story Points: 5

Labels: backend, security, infrastructure

Description:

As a security stakeholder, I need proper security headers and CORS policies to protect against common web attacks.

Acceptance Criteria:

- Configure security headers:
 - Content-Security-Policy
 - Strict-Transport-Security
 - X-Frame-Options
 - X-Content-Type-Options
 - Referrer-Policy
- Configure CORS:
 - Allowed origins
 - Allowed methods
 - Credentials policy
- Test header configuration

Technical Requirements:

- Next.js middleware for headers
- CORS configuration
- Header testing tools

Implementation Tasks:

1. Define security headers
 2. Configure in Next.js
 3. Set up CORS policies
 4. Test with browser DevTools
 5. Document configuration
-

Story 6.5: PII Protection & Compliance

Priority: P0 (Critical)
Story Points: 8
Labels: backend, security, compliance

Description:
As a compliance officer, I need PII properly handled and masked in logs to comply with GDPR and CCPA requirements.

Acceptance Criteria:

- Mask PII in logs:
 - Emails: [user@example.com](#) → u***@example.com
 - Names: John Smith → J*** S***
 - Phone numbers
- Secure session cookies (httpOnly, secure, sameSite)
- Data retention policies
- GDPR-compliant consent flow
- Right to deletion support

Technical Requirements:

- PII masking utility
- Secure cookie configuration
- Data retention automation
- Consent management

Implementation Tasks:

1. Create PII masking utility
2. Configure secure cookies
3. Define retention policies
4. Implement consent flow
5. Add deletion endpoints
6. Test compliance
7. Document policies

Epic 7: Observability & Analytics

Story 7.1: Structured Logging System

Priority: P0 (Critical)
Story Points: 13
Labels: backend, infrastructure, observability

Description:
As a developer, I need structured logging throughout the application to debug issues and monitor system health.

Acceptance Criteria:

- JSON-formatted logs
- Log levels: debug, info, warn, error
- Include context: session_id, route, user_agent
- Log all Agent calls with:
 - Prompt and response (PII masked)
 - Latency and tokens

- Cost estimation
- Log page generations
- Log errors with stack traces
- Log performance metrics

Technical Requirements:

- Logging solution (Vercel/Datadog/LogRocket)
- Log schema definition
- Log aggregation
- Searchable logs

Implementation Tasks:

1. Choose logging solution
 2. Define log schema
 3. Implement logger utility
 4. Add logging throughout app
 5. Add PII masking
 6. Configure log aggregation
 7. Test log output
 8. Create debugging queries
-

Story 7.2: Error Tracking & Alerting

Priority: P0 (Critical)

Story Points: 8

Labels: backend, infrastructure, observability

Description:

As an operations stakeholder, I need automatic error detection and alerting to respond quickly to production issues.

Acceptance Criteria:

- Capture all unhandled errors
- Include stack traces with source maps
- Group similar errors
- Alert on new error types
- Alert on error rate spikes
- Integration with Slack/email
- Track error resolution

Technical Requirements:

- Error tracking service (Sentry/Rollbar)
- Alert configuration
- Source map upload

Implementation Tasks:

1. Choose error tracking service
 2. Integrate into application
 3. Configure error grouping
 4. Set up alerting rules
 5. Configure notifications
 6. Upload source maps
 7. Test error capture
-

Story 7.3: Performance Monitoring

Priority: P1 (High)

Story Points: 8

Labels: frontend, backend, observability

Description:

As a performance engineer, I need to track Core Web Vitals and custom metrics to ensure excellent user experience.

Acceptance Criteria:

- Track Core Web Vitals:
 - LCP < 2.5s
 - FID < 100ms
 - CLS < 0.1
- Track custom metrics:
 - Page generation time
 - Component render time
 - Agent response time
- Real User Monitoring (RUM)
- Alert on metric degradation

Technical Requirements:

- Performance monitoring solution
- Web Vitals API
- Custom metric collection

Implementation Tasks:

1. Implement Web Vitals tracking
2. Add custom metric collection
3. Configure monitoring dashboard
4. Set performance budgets
5. Add alerting
6. Test metric accuracy
7. Document metrics

Story 7.4: Analytics Dashboard

Priority: P1 (High)

Story Points: 13

Labels: frontend, analytics, business-intelligence

Description:

As a product stakeholder, I want an analytics dashboard to understand user behavior and system performance.

Acceptance Criteria:

- Conversion funnel visualization
- Top viewed features
- Persona distribution
- Performance metrics (P50, P95, P99)
- Agent cost tracking
- Session duration analysis
- Question analysis (common themes)
- Date range filtering
- Export capability

Technical Requirements:

- Analytics data warehouse
- Visualization library (Recharts)
- Data aggregation queries
- Real-time or near-real-time updates

Implementation Tasks:

1. Design dashboard layout
2. Build data aggregation pipeline
3. Create visualization components
4. Implement filters
5. Add export functionality
6. Optimize for performance
7. Add drill-down capabilities
8. Test with production data

Epic 8: Performance & Optimization

Story 8.1: Page Generation Optimization

Priority: P1 (High)

Story Points: 8

Labels: backend, performance, ai

Description:

As a user, I want pages generated quickly (< 2 seconds) so I don't have to wait during my exploration.

Acceptance Criteria:

- Page generation < 2s (P90)
- Page generation < 3s (P99)
- Streaming responses for faster perceived performance
- Cache common page patterns
- Parallel processing where possible
- Graceful degradation for slow responses

Technical Requirements:

- Response streaming
- Caching strategy
- Performance profiling
- Optimization opportunities

Implementation Tasks:

1. Profile current generation performance
2. Implement response streaming
3. Add caching for common patterns
4. Optimize component selection
5. Parallelize independent operations
6. Add performance monitoring
7. Load test and optimize
8. Document optimization strategies

Story 8.2: Component Loading Strategy

Priority: P1 (High)

Story Points: 5

Labels: frontend, performance

Description:

As a user, I want pages to load quickly without large bundle sizes that slow down initial page load.

Acceptance Criteria:

- Code splitting by component
- Lazy loading for heavy components
- Initial bundle < 500KB
- TTI < 3 seconds
- Preload common components
- Progressive enhancement

Technical Requirements:

- Next.js code splitting
- React lazy loading
- Bundle analysis
- Performance budgets

Implementation Tasks:

1. Analyze current bundle size
 2. Implement code splitting
 3. Add lazy loading
 4. Identify critical components
 5. Preload strategy
 6. Measure impact
 7. Document loading strategy
-

Story 8.3: Knowledge Base Query Optimization

Priority: P1 (High)

Story Points: 5

Labels: backend, performance, rag

Description:

As the system, I need fast knowledge base queries (< 500ms) to provide responsive user experiences.

Acceptance Criteria:

- Vector search < 500ms (P95)
- Optimize index for query patterns
- Cache frequent queries
- Implement query result pagination
- Monitor query performance

Technical Requirements:

- Vector index optimization
- Query caching (Redis)
- Performance monitoring
- Query profiling

Implementation Tasks:

- 1. Profile current query performance
 - 2. Optimize vector indexes
 - 3. Implement query caching
 - 4. Add pagination
 - 5. Test with load
 - 6. Monitor in production
 - 7. Document optimization
-

Story 8.4: Image & Asset Optimization

Priority: P2 (Medium)
Story Points: 5
Labels: frontend, performance, infrastructure

Description:
As a user, I want images and assets to load quickly without consuming excessive bandwidth.

Acceptance Criteria:

- Serve images in modern formats (WebP, AVIF)
- Implement responsive images
- Use CDN for asset delivery
- Lazy load images
- Compress images automatically
- Set proper cache headers

Technical Requirements:

- Next.js Image component
- CDN configuration
- Image optimization service

Implementation Tasks:

- 1. Audit current image usage
 - 2. Implement Next.js Image
 - 3. Configure CDN
 - 4. Add lazy loading
 - 5. Set up automatic compression
 - 6. Configure cache headers
 - 7. Measure impact
-

Story 8.5: Database Query Optimization

Priority: P1 (High)
Story Points: 8
Labels: backend, performance, database

Description:
As a system administrator, I need optimized database queries to support application scale and performance.

Acceptance Criteria:

- All queries < 100ms (P95)
- Proper indexing strategy
- Connection pooling

- Query result caching
- N+1 query prevention
- Monitor slow queries

Technical Requirements:

- Database indexes
- Connection pooling (PgBouncer)
- Query caching
- Performance monitoring

Implementation Tasks:

1. Audit current queries
2. Add missing indexes
3. Implement connection pooling
4. Add query result caching
5. Refactor N+1 queries
6. Monitor query performance
7. Document optimization

Implementation Phases

Phase 1: Foundation (Weeks 1-4)

Goal: Core chat and page generation working end-to-end

Stories:

- 1.1: Build Persistent Chat Interface
- 1.2: Implement Intent Classification System
- 1.3: Build Dynamic Page Generator
- 1.4: Implement Page Renderer
- 1.5: Context Management System
- 3.1: Core Content Components (subset)
- 6.1: Input Validation & Sanitization
- 7.1: Structured Logging System

Success Criteria:

- User can ask questions and receive dynamic pages
- Basic components render correctly
- Conversation context maintained
- All inputs validated
- Logging operational

Phase 2: Intelligence (Weeks 5-8)

Goal: Add RAG knowledge base and improve response quality

Stories:

- 2.1: Vector Database Setup
- 2.2: Content Embedding Pipeline
- 2.3: RAG Retrieval System

- 2.5: Initial Knowledge Base Population
- 3.1: Core Content Components (complete)
- 3.2: Data Visualization Components
- 4.1: Persona Classification System
- 7.2: Error Tracking & Alerting

Success Criteria:

- Agent uses product knowledge for responses
 - All core components available
 - Basic persona detection working
 - Error tracking operational
-

Phase 3: Engagement (Weeks 9-12)

Goal: Add lead capture, CRM integration, and personalization

Stories:

- 4.2: Content Personalization Engine
- 5.1: Email Capture Flow
- 5.2: CRM Integration
- 5.3: Conversation Summary Generation
- 3.3: Interactive Components
- 6.2: Rate Limiting Implementation
- 6.3: Prompt Injection Protection

Success Criteria:

- Leads captured and synced to CRM
 - Personalized experiences by persona
 - Interactive components functional
 - Security measures in place
-

Phase 4: Scale & Polish (Weeks 13-16)

Goal: Optimize performance, add advanced features, production hardening

Stories:

- 2.4: CMS to Vector DB Reindex Pipeline
- 4.3: Persona Analytics Dashboard
- 6.4: Security Headers & CORS
- 6.5: PII Protection & Compliance
- 7.3: Performance Monitoring
- 7.4: Analytics Dashboard
- 8.1: Page Generation Optimization
- 8.2: Component Loading Strategy
- 8.3: Knowledge Base Query Optimization
- 8.4: Image & Asset Optimization
- 8.5: Database Query Optimization

Success Criteria:

- Content automatically syncs from CMS
- Performance targets met
- Security audit passed
- Analytics fully operational

- Production-ready
-

Success Metrics

Technical Metrics

- Page generation time: < 2s (P90), < 3s (P99)
- Component render time: < 1s (P95)
- Knowledge base query: < 500ms (P95)
- System uptime: > 99.9%
- Error rate: < 0.1%

Business Metrics

- Email capture rate: > 20%
- CRM sync success: > 99%
- Session duration: > 3 minutes (engaged users)
- Pages per session: > 3
- Bounce rate: < 40%

Quality Metrics

- Intent classification accuracy: > 90%
 - Component selection appropriateness: > 90%
 - Knowledge retrieval relevance: > 85%
 - User satisfaction: > 4.0/5.0
-

Technical Stack

Frontend

- **Framework:** Next.js 14+ (App Router)
- **Language:** TypeScript 5+
- **UI Library:** React 18+
- **Component Library:** shadcn/ui
- **Styling:** Tailwind CSS
- **Charts:** Recharts
- **State Management:** Zustand or Context API
- **Forms:** React Hook Form + Zod

Backend

- **Framework:** Next.js API Routes & Server Actions
- **Language:** TypeScript
- **AI/LLM:** Anthropic Claude / OpenAI GPT-4
- **Vector Database:** Supabase pgvector or Pinecone
- **Primary Database:** PostgreSQL (Supabase)
- **Caching:** Redis (Upstash)
- **CRM:** HubSpot or Salesforce
- **Email:** SendGrid or Postmark

Infrastructure

- **Hosting:** Vercel

- **Monitoring:** Vercel Analytics + Sentry
 - **Logging:** Vercel Logs or Datadog
 - **CDN:** Vercel Edge Network
 - **Rate Limiting:** Upstash Rate Limit
-

Constraints & Assumptions

Constraints

1. Must work in Claude.ai artifact environment (no localStorage)
2. No external libraries beyond approved list
3. Must be mobile-responsive
4. Must meet WCAG 2.1 AA accessibility
5. Must comply with GDPR and CCPA

Assumptions

1. Users have modern browsers (last 2 versions)
 2. Average session duration: 5-10 minutes
 3. Concurrent users: 100 (Phase 1), 1,000 (Phase 2), 10,000 (Phase 3)
 4. Knowledge base: ~10,000 content chunks
 5. LLM costs: ~\$0.01 per page generation
-

Next Steps

1. **Review & Approve:** Stakeholder review of this PRD
 2. **Resource Allocation:** Assign team members to epics
 3. **Sprint Planning:** Break stories into sprint-sized tasks
 4. **Environment Setup:** Provision infrastructure and services
 5. **Kickoff:** Phase 1 development begins
-

Document Control

Version History:

- v2.0 (2025-10-09): Integrated PRD with implementation tickets
- v1.0 (2025-10-08): Initial PRD draft

Approvals:

- Product: [Pending]
- Engineering: [Pending]
- Security: [Pending]
- Design: [Pending]

Related Documents:

- Technical Architecture Document
- API Specification
- Component Design System
- Security Review
- Compliance Checklist